

EXERCISE – 6

ERROR DETECTION AND CORRECTION USING HAMMING CODE

AIM

To write a program to implement error detection and correction using the Hamming code concept, input a text data stream, add redundant bits, transmit the encoded data through a channel file, and verify the error correction feature.

PROCEDURE

1. Create the sender program.
2. Read the input text from `input.txt`.
3. Convert each character of the text into its binary representation.
4. Apply Hamming code and add the required redundant bits.
5. Store the encoded binary data in a file named `channel.txt`.
6. Create the receiver program and read the data from `channel.txt`.
7. Calculate the Hamming syndrome to detect the error position.
8. If an error is detected, correct the corresponding bit.
9. Remove the redundant bits from the corrected data.
10. Convert the remaining binary data into ASCII characters and display the original text.

The sender and receiver requirements follow the lab manual's Practical-6 specification.

SENDER CODE

```
#include <stdio.h>
#include <string.h>

void encodeCharacter(unsigned char data, FILE *channel)
{
    int h[13] = {0};
    int dataBit = 0;

    for (int i = 1; i <= 12; i++)
    {
        if (i != 1 && i != 2 && i != 4 && i != 8)
        {
            h[i] = (data >> dataBit) & 1;
            dataBit++;
        }
    }
}
```

```

    }

    h[1] = h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
    h[2] = h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
    h[4] = h[5] ^ h[6] ^ h[7] ^ h[12];
    h[8] = h[9] ^ h[10] ^ h[11] ^ h[12];

    for (int i = 1; i <= 12; i++)
        fprintf(channel, "%d", h[i]);

    fprintf(channel, "\n");
}

int main()
{
    FILE *input;
    FILE *channel;
    char ch;

    input = fopen("input.txt", "r");
    channel = fopen("channel.txt", "w");

    if (input == NULL || channel == NULL)
    {
        printf("File opening error\n");
        return 1;
    }

    while ((ch = fgetc(input)) != EOF)
        encodeCharacter((unsigned char)ch, channel);

    fclose(input);
    fclose(channel);

    printf("Data encoded and stored in channel.txt\n");

    return 0;
}

```

RECEIVER CODE

```

#include <stdio.h>
#include <string.h>

int decodeCharacter(char word[], int *errorPosition)

```

```

{
    int h[13] = {0};
    int syndrome = 0;
    int data = 0;
    int bit = 0;

    for (int i = 1; i <= 12; i++)
        h[i] = word[i - 1] - '0';

    int p1 = h[1] ^ h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
    int p2 = h[2] ^ h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
    int p4 = h[4] ^ h[5] ^ h[6] ^ h[7] ^ h[12];
    int p8 = h[8] ^ h[9] ^ h[10] ^ h[11] ^ h[12];

    syndrome = p1 + (p2 * 2) + (p4 * 4) + (p8 * 8);

    *errorPosition = syndrome;

    if (syndrome >= 1 && syndrome <= 12)
        h[syndrome] ^= 1;

    for (int i = 1; i <= 12; i++)
    {
        if (i != 1 && i != 2 && i != 4 && i != 8)
        {
            data = data | (h[i] << bit);
            bit++;
        }
    }

    return data;
}

int main()
{
    FILE *channel;
    char word[20];
    int errorPosition;
    int data;
    int block = 1;

```

```

channel = fopen("channel.txt", "r");

if (channel == NULL)
{
    printf("Unable to open channel.txt\n");
    return 1;
}

printf("Decoded Output:\n");

while (fscanf(channel, "%12s", word) == 1)
{
    data = decodeCharacter(word, &errorPosition);

    if (errorPosition != 0)
        printf("\nError in block %d at position %d\n",
            block, errorPosition);

    printf("%c", (char)data);
    block++;
}

fclose(channel);

printf("\n");

return 0;
}

```

OUTPUT

INPUT

Input file: **input.txt**

HELLO

OUTPUT

Sender Output:

Data encoded and stored in channel.txt

Receiver Output:

Decoded Output:

HELLO

Error Test Output

Decoded Output:

Error in block 2 at position 5

HELLO

RESULT

The Hamming code error detection and correction program was successfully implemented and tested.